



RELATÓRIO TÉCNICO - TRABALHO PRÁTICO: Simulador Protocolo MESI em uma Aplicação

Discente:

Vinícius Schmitz Freitas de Oliveira

RA:

109887

Docente:

Sandra Cossul

Disciplina:

Arquitetura e Organização de Computadores II

Maringá, 31 de Agosto de 2024

1. SUMÁRIO

1. Sumário	1
2. Introdução	2
3. Objetivos e Justificativas	3
3.1 <i>Objetivos</i>	
3.2 <i>Justificativas</i>	
4. Funcionamento da Cache	6
5. Funcionamento do Protocolo MESI	7
6. Funcionamento da Aplicação	10
7. Decisões de Projeto	11
7.1 <i>Estruturas de Dados utilizadas</i>	
7.2 <i>Funções e Gerenciamento do Protocolo MESI</i>	
8. Conclusão	17
9. Referências	18

2. INTRODUÇÃO

A memória cache é um dos elementos mais importantes na arquitetura de computadores, ela possibilita um acréscimo significativo de desempenho utilizando do princípio da localidade. Estando no segundo nível mais alto na hierarquia de memória (só ficando atrás dos registradores), ela tem um papel importante em relação à busca de dados pelo processador pois serve como um espaço de armazenamento temporário de dados que são utilizados frequentemente ou que estão próximos uns dos outros (princípio da localidade) na memória principal.

Este trabalho terá como foco principal a construção de um simulador de 3 memórias caches junto com a implementação do gerenciamento entre elas utilizando o protocolo MESI, para ser mantida a coerência dos dados entre as caches.

3. OBJETIVOS E JUSTIFICATIVAS

3.1 Objetivos

O objetivo deste trabalho inicialmente é implementar em uma aplicação real A SER DECIDIDA AINDA uma memória cache gerenciada pelo protocolo MESI (Modify, Exclusive, Shared e Invalid). O programa também deverá conter:

- ☒ **Memória Cache com 5 posições**
- ☒ **Memória Principal (RAM) com 50 posições**
- ☒ **3 Processadores cada um com 1 nível de cache dedicada**
- ☒ **2 Espaços de endereçamento:**
 - ☒ Um para simular a memória principal;
 - ☒ Outro para simular a memória cache alocada a cada 1 das CPU 's.

Funcionamento do programa:

- ☒ **0 programa deverá solicitar acessos aos dados contidos na memória principal.**
 - ☒ Tamanho de bloco da memória RAM deve ser levado em conta, pois cada um é alocado a uma linha da cache;
 - ☒ As solicitações podem ocorrer em cada um dos processadores, de forma independente entre eles;

Memória principal	
Linha	Dado
0	3434
1	24121
2	232
3	5028
4	81092
5	91639
6	54432
...	...

Memória cache – P1	Tags – P1

Memória cache – P2	Tags – P2

Inicialização:

- ☒ **No início o programa deve preencher o espaço de memória principal com valores aleatórios:**
 - ☒ Numéricos ou Alfanuméricos;
 - ☐ Outras informações de acordo com o tipo de aplicação escolhida.
- ☒ **Em seguida, o programa deve possibilitar ao usuário selecionar um processador para realizar a solicitação de um dado da memória principal:**
 - ☒ Solicitação com alteração do valor (Escrita);
 - ☒ Solicitação sem alteração do valor (Leitura).

Mapeamento da cache:

- ☒ **A função de mapeamento da cache deve preencher as linhas em ordem aleatória:**
 - ☒ Algoritmo de substituição deve ser o *FIFO*;
 - ☒ Política de escrita deve ser a *WRITE-BACK*.

- ☑ Quando a solicitação é realizada, cada linha da cache pode estar associada aos valores de acordo com o protocolo *MESI* e esta informação deve estar visível na tela ao ser mostrada a representação da memória cache:
 - ☑ Uma linha da cache é **modificada**, quando seu valor foi alterado na cache e ainda permanece inalterado na RAM;
 - ☑ Uma linha da cache é **exclusiva** quando somente um processador está acessando a posição da memória RAM;
 - ☑ Uma linha da cache é **compartilhada** se mais de um processador está acessando a posição da memória RAM;
 - ☑ Uma linha da cache é **inválida** se outro processador alterou o valor da posição da memória RAM.

- ☑ Os seguintes tipos de transações devem estar definidos:
 - ☑ **RH (Read hit - leitura com acerto)**, quando a leitura de um dado é solicitada e este já está na cache do processador solicitante;
 - O estado é mantido;
 - Se está modificado fica modificado, se está compartilhado fica compartilhado, e se está exclusivo fica exclusivo.
 - ☑ **RM (Read Miss - leitura com falha)**, quando a leitura de um dado é solicitada e este não está na cache do processador solicitante;
 - Se o dado está presente em outra cache, a linha é marcada como compartilhada;
 - Senão é marcada como exclusiva.
 - ☑ **WM (Write Miss - escrita com falha)**, quando a escrita de um dado é solicitada e este não está na cache do processador solicitante;
 - Após a linha ser carregada é marcada como modificada;
 - Se a linha estiver presente em outra cache, é marcada como inválida na outra.

- ☑ **WH (*Write Hit* - escrita com acerto)**, quando a escrita de um dado é solicitada e está na cache do processador solicitante.
 - Se a linha é compartilhada muda-se para modificada e todas as outras para inválida;
 - Se a linha é exclusiva muda-se para modificada;
 - E se a linha já está marcada como modificada seu estado é mantido.

- ☑ **Ao executar qualquer solicitação na memória, deve ser destacado se ocorreu um *RH*, *RM*, *WM* ou *WH*. Destacando ainda o estado da linha.**

- ☑ **O campo *TAGS* da memória cache deve armazenar as informações relevante sobre a linha, como:**
 - ☑ A posição correspondente da memória RAM (bloco);
 - ☑ E as informações referentes ao protocolo MESI;
 - ☑ Além disso, estas informações **NÃO** devem estar armazenadas na memória RAM, somente na cache.

3.2 Justificativas

O programa não simula nenhuma aplicação, então os dados da RAM e que ficam contidos nas linhas das caches são apenas números aleatórios que estão entre [0-9999]. Isto porque o foco foi principalmente fazer o gerenciamento do protocolo de coerência de cache MESI, dito isto, é importante ressaltar que em compensação foram utilizadas *threads* simulando os processadores o que possibilita a incrementação do programa para simular algum tipo de aplicação real que as utiliza para fazer o processamento de forma concorrente. Mas isto fica para uma outra versão.

4. FUNCIONAMENTO DA CACHE

Uma memória cache basicamente serve para possibilitar que a CPU busque um recurso de forma mais rápida pelo fato do acesso à cache ser menos custoso do que acessar a memória principal, isso utilizando o **princípio da localidade** temporal e espacial. É encontrada no processador e possui a característica de ser volátil, assim, perdendo os dados armazenados quando é desprovida de energia.

A cache pode funcionar com tipos de mapeamentos diferentes, o abordado neste trabalho será o mapeamento associativo. Nele a RAM tem seus endereços agrupados em blocos, estes que são o conteúdo das linhas da cache. Quando o processador buscar uma palavra primeiramente ele irá verificar se o bloco dessa palavra está na cache, se sim temos um **cache hit** e não há necessidade de busca da palavra na RAM, o que gera uma economia de tempo significativa no desempenho. Mas se a palavra não estiver na cache, então temos um **cache miss** fazendo com que seja necessário buscar a palavra na RAM e então colocá-la na cache, como usaremos o mapeamento associativo temos que qualquer bloco da RAM pode estar em qualquer linha da cache mas como decidir qual linha será substituída? Nesse caso devemos escolher uma política de substituição, aqui usaremos a clássica **FIFO** (First-in First-out) onde a linha da cache que está há mais tempo armazenada é substituída.

Dessa forma, a cache é um dos elementos mais essenciais para o funcionamento de computadores. Mas com isso surge um problema, a necessidade de manter a coerência dos dados quando se tem sistemas multicore com vários processadores cada um com sua(s) cache(s), dado isso como é possível gerenciar as diferentes caches (estados locais) com a RAM (estado global)? Utilizando protocolos de coerência de cache, a seguir será explicado o utilizado no trabalho, o protocolo de monitoramento (*snoopy*) com abordagem *write invalidate*, o protocolo MESI.

5. FUNCIONAMENTO DO PROTOCOLO MESI

Protocolo de coerência de cache do tipo protocolo de monitoramento (*snoopy*) com abordagem *write invalidate*, sendo baseado em estados onde cada um representa uma condição específica dos dados em relação à memória principal e a outras caches.

Estados: A cache pode incluir 2 bits de estado para cada *tag*, possibilitando cada linha estar em um desses estados →

1. **Modificada**: linha da cache foi modificada (diferente da memória principal) e disponível apenas nessa cache
2. **Exclusiva**: linha na cache é a mesma da memória principal e não está presente em nenhuma outra cache
3. **Compartilhada**: linha na cache é a mesma da memória principal e pode estar presente em outra cache
4. **Inválida**: linha na cache não contém dados válidos

Operações:

1. **Read Miss**: quando ocorre uma falha de leitura em uma cache local, o CPU inicia uma leitura de memória para ler a linha da memória principal que contém o endereço que está faltando, então ele insere um sinal no barramento que avisa todos os outros CPUs para monitorarem a operação. Então existem 4 possíveis desfechos →
 - Se outra cache possui uma cópia limpa (não modificada desde a leitura da memória) da linha no estado EXCLUSIVO, ela retorna um sinal indicando que compartilha essa linha. Após, passa o estado da sua cópia para COMPARTILHADA. O CPU que requisitou lê a linha do barramento e atualiza a linha na sua cache de INVÁLIDA para COMPARTILHADA.
 - Se uma ou mais caches têm uma cópia limpa da linha no estado COMPARTILHADO, cada uma delas sinaliza que compartilha essa linha. O CPU que requisitou lê a linha do barramento e passa-a na sua cache de INVÁLIDA para COMPARTILHADA.

- Se outra cache tem uma cópia MODIFICADA da linha, então essa cache bloqueia a leitura de memória e fornece a linha para a cache que requisitou por meio do barramento compartilhado, atualizando a linha de MODIFICADA para COMPARTILHADA. A linha enviada para a cache requisitante é também recebida e processada pelo controlador de memória, que guarda o bloco na memória.
- Se nenhuma outra cache tem uma cópia da linha (limpa ou MODIFICADA), então nenhum sinal é retornado. O CPU que requisitou lê a linha e passa-a na sua cache de INVÁLIDA para EXCLUSIVA.

2. Read Hit: quando uma leitura com acerto ocorre em uma linha que está eventualmente na cache local, o processador simplesmente lê o item requerido. Não havendo mudança de estado, permanecendo MODIFICADO, COMPARTILHADO ou EXCLUSIVO.

3. Write Miss: quando ocorre uma escrita com falha na cache local, o processador inicia uma leitura de memória para ler a linha da memória principal contendo o endereço que falhou. Então ele emite um sinal no barramento que significa LEITURA COM INTENÇÃO DE MODIFICAR (RWITM) e quando a linha é carregada é imediatamente marcada como MODIFICADA. Em relação a outras cache, 2 cenários possíveis antecedem o carregar da linha de dados →

- Outra cache pode ter uma cópia modificada dessa linha (estado = MODIFICADO):
 - o CPU alertado sinaliza ao CPU iniciante que outro CPU tem uma cópia modificada da linha
 - o CPU que iniciou entrega o barramento e espera
 - o outro CPU que obtém acesso ao barramento escreve na linha de cache modificada de volta na memória principal e passa o estado da linha de cache para INVÁLIDA (pq o CPU que iniciou vai modificar essa linha)

- o CPU que iniciou emite novamente uma sinal RWITM para o barramento e depois lê a linha da memória principal, modifica a linha na cache e marca a linha para o estado MODIFICADO

- Nenhuma outra cache possui uma cópia modificada da linha requisitada:

- nenhum sinal é retornado e o CPU iniciante continua a ler a linha e modificá-la
- se uma ou mais caches possuem uma cópia limpa da linha no estado COMPARTILHADO, cada cache invalida a sua cópia da linha
- e se uma cache tiver uma cópia da linha no estado EXCLUSIVO, ela invalida a sua cópia da linha

4. Write Hit: Quando ocorre uma escrita com sucesso em uma linha que está atualmente na cache local, o efeito depende do estado atual dessa linha na cache local →

- COMPARTILHADA:

- antes de efetuar a atualização, o CPU deve obter a prioridade exclusiva da linha
- o CPU sinaliza a sua intenção no barramento
- todo CPU que tem uma cópia compartilhada da linha na sua cache passa-a de compartilhada para inválida
- o CPU que iniciou efetua então a atualização e passa a sua cópia da linha de COMPARTILHADA para MODIFICADA

- EXCLUSIVA:

- o CPU já possui o controle exclusivo dessa linha, então ele simplesmente efetua a atualização e passa a sua cópia da linha de exclusiva para MODIFICADA

- MODIFICADA:

- o CPU já possui o controle exclusivo dessa linha e a linha está marcada como MODIFICADA, então ele simplesmente efetua a atualização

6. FUNCIONAMENTO DA APLICAÇÃO

Visão Geral:

O programa desenvolvido, basicamente, apenas simula os 3 processadores, cada um com sua respectiva cache. Para isso foi utilizado o conceito de *threads* onde cada uma simula um dos processadores, sendo possível escolher entre um deles para fazer uma solicitação de leitura ou escrita em cima de um endereço da memória principal, endereço este que é inserido pelo usuário.

Inicialização:

O programa inicializa a RAM chamando `initRam(RAM *ram)` com números aleatórios, gerados por `srand(time(NULL))`, e as caches chamando `initCache(Cache *cache, int id)` com valores -1 para seus campos e **I (Invalid)** para o campo de estado MESI. É mostrado o estado geral das estruturas. E então as **threads** são criadas e é atribuído à elas os respectivos **ids**, após isso é perguntado ao usuário qual processador ele deseja utilizar e de acordo com a resposta a **thread** correspondente é disparada.

Protocolo MESI:

Após isso, inicia-se a função `*simula(void *ptr)` que simula o protocolo MESI, nela é perguntado se o usuário quer fazer uma solicitação de dado s/ alteração do valor (de leitura) ou c/ alteração do valor (de escrita), no caso da escolha ser:

- **Leitura:** Função `leitura(int idCache, int enderecoRam)`
- **Escrita:** Função `escrita(int idCache, int enderecoRam)`

Nelas são feitas as operações para ler ou sobrescrever um dado contido em um bloco na RAM, este que pode ter cópias na cache do processador atual, nas caches vizinhas (em apenas uma ou ambas) ou em nenhum lugar. Aqui temos as principais operações do protocolo:

- **Read Hit & Read Miss**
- **Write Hit & Write Miss**

Em ambos os tipos de operações, são utilizadas as 3 principais funções do código:

- *procuraCopias(int idCache, int *indCopias, int bloco)*
- *leDaRam(int linha, int idCache, int bloco, int dado)*
- *leDoVizinho(int linha, int idCache, int LinhaDoVizinho, int idVizinho, int bloco, int dado)*

7. DECISÕES DE PROJETO

7.1 Estruturas De Dados Utilizadas

DEFINIÇÕES:

- **#include <time.h>**
 - o biblioteca para geração de números aleatórios
- **#include <pthread.h>**
 - o biblioteca para uso de *threads*
- **#define RAM_TAM 50**
- **#define BLOCO_TAM 2**
- **#define CACHE_TAM 5**
- **#define CPU_QNT 3**
 - o tamanhos da RAM, blocos, caches e quantidade de CPUs; respectivamente
- **typedef struct**
 - {
 - int id;
 - } **tPacote;**
 - o pacote das threads, só contém id nesta versão

RAM:

- **typedef struct**
 - {
 - int dado[RAM_TAM];
 - } **Ram;**
 - representa a memória principal, contém vetor de inteiros com RAM_TAM posições

CACHE:

- **typedef struct**
 - {
 - int posBloco;
 - char estado;
 - } **Tag;**
 - representa a tag das linhas das caches, contém um inteiro indicando posição do bloco na RAM e um caracter indicando estado MESI (Modified, Exclusive, Shared, Invalid)
- **typedef struct**
 - {
 - int dado[BLOCO_TAM];
 - } **Bloco**
 - representa o conteúdo dos blocos, contém vetor de inteiros com BLOCO_TAM posições (2 dados apenas)
- **typedef struct**
 - {
 - Tag tag;
 - Bloco bloco;
 - } **cacheLinha;**
 - representa as linhas das caches, contém a tag e o bloco

- **typedef struct**
 - {
 - cacheLinha linha[CACHE_TAM];
 - int *fifoFila,
 - fifoQnt;
 - int preenchida;
 - } **Cache;**
 - representa as caches, contém um vetor de structs (do tipo cacheLinha) de CACHE_TAM posições, ponteiro pra inteiro apontando para uma fila que gerencia a substituição das linhas por FIFO, um inteiro para a quantidade de elementos contidos na fila FIFO e um inteiro para verificar se a cache está preenchida já

7.2 Funções e Gerenciamento do Protocolo MESI

Principais Funções:

As principais funções do código, já citadas anteriormente.

- **void *simula(void *ptr):** do tipo ponteiro para void, é executada pelas threads e leva como argumento a struct da thread do tipo *tPacote*, exige do usuário escolher se ele deseja uma leitura ou escrita, e um endereço da memória principal para ser feita a operação
- **int leitura(int idCache, int enderecoRam):** do tipo int, leva como argumentos a id da cache atual e o endereço da ram digitado pelo usuário. Primeiro ela verifica se o dado está na cache do CPU atual percorrendo suas linhas, se:
 - sim, indica Read Hit e imprime a linha
 - não, verifica (com *switch case*) se existem cópias em outras caches utilizando a função *procuraCopias()* e o vetor *indiceCopias[3]* que guarda as linhas onde se

encontram as cópias ou -1 caso não haja nenhuma, então se:

- não (0), indica Read Miss e copia o bloco da RAM para a linha da cache
- sim uma (1), verifica (com outro *switch case*) em qual das caches vizinhas a cópia está e então a copia para a sua cache. Se a cópia é E - Exclusive apenas troca os estados de ambas para S - Shared, se é M - Modified troca os estados para S e atualiza a memória principal
- sim duas (2), escolhe entre uma das caches vizinhas para copiar para a sua e apenas atualiza a sua para S - Shared

- **int escrita(int idCache, int enderecoRam):** do tipo int, assim como a anterior, leva como argumentos a id da cache atual e o endereço da ram digitado pelo usuário. Primeiro ela verifica se o dado está na cache do CPU atual percorrendo suas linhas, se:

- sim, verifica o estado da linha:
 - M - Modified, indica Write Hit e apenas pede ao usuário um valor novo para o dado solicitado e o atualiza (apenas o da cache, mantendo o da RAM desatualizado) e mantém o estado M
 - E - Exclusive, indica Write Hit e pede o valor e atualiza os dados, e altera o estado para M
 - S - Shared, indica Write Hit e invalida a(s) cópia(s), verifica (com *switch case*) se possui 1 ou 2 cópias usando a função *procuraCopias()* e o vetor *indiceCopias[3]*
- não, verifica se há cópias (com *switch case*) da mesma forma, se:
 - 0 cópias, indica Write Miss, copia o bloco da RAM e atualiza o estado para M

- 1 cópia, verifica em qual cache está, e se o estado for:
 - M, atualiza a RAM com o conteúdo da cópia, invalida, lê a cópia atualizada da RAM, pede ao usuário valor novo, indica Write Miss, atualiza e altera estado para M
 - E, apenas a cópia diretamente, atualiza com o valor novo inserido, indica Write Miss e altera estado para M
- 2 cópias, invalida ambas as cópias, lê da RAM (aqui vai uma observação: no funcionamento de uma cache real acho que nesse caso o valor a ser alterado seria lido de uma das caches invalidadas, porém mantive por ler da ram mesmo por conta de não precisar alterar a função `leDoVizinho` que explicarei a seguir), atualiza com valor novo inserido, indica Write Miss e altera estado para M

- **`void leDoVizinho(int linha, int idCache, int linhaVizinho, int idVizinho, int bloco, int dado):`** do tipo `void`, leva os argumentos a linha da cache atual a ser substituída, a id da cache atual, a linha do vizinho aonde se encontra a cópia, a id da cache do vizinho, o bloco a ser copiado e o *offset* do dado no bloco. Primeiro ela verifica pela variável *preenchida* se a cache já está preenchida, se:
 - não (0), atribui o bloco a uma linha aleatória da cache, e verifica se ela já possui um bloco inserido, se:
 - não, copia o bloco da cache vizinha, atualiza a fila FIFO, altera o estado de ambas para S - Shared e indica Read Miss
 - sim, ela pula para a próxima linha até encontrar uma vazia, então copia o bloco da cache vizinha, atualiza a fila FIFO, altera o estado de ambas para S e indica Read Miss

- sim (1), verifica se não há nenhuma linha inválida para substituí-la e aproveitar melhor o espaço da cache, se:
 - não, atualiza a fila, substitui a linha que entrou primeiro (FIFO - First In First Out), altera os estados para S e indica Read Miss
 - sim, atualiza a fila de acordo, e a substitui invés de utilizar FIFO, altera os estados para S e indica Read Miss
- **int leDaRam(int linha, int idCache, int bloco, int dado):**
 - do tipo *int*, leva os argumentos a linha da cache atual a ser substituída, a id da cache atual, o bloco a ser copiado e o *offset* do dado no bloco. Primeiro ela verifica pela variável *preenchida* se a cache já está preenchida, se:
 - não (0), atribui o bloco a uma linha aleatória da cache, e verifica se ela já possui um bloco inserido, se:
 - não, copia o bloco da RAM, atualiza a fila FIFO, altera o estado para E - Exclusive
 - sim, ela pula para a próxima linha até encontrar uma vazia, então copia o bloco da RAM, atualiza a fila FIFO e altera o estado para E
 - sim (1), verifica se não há nenhuma linha inválida para substituí-la e aproveitar melhor o espaço da cache, se:
 - não, atualiza a fila, substitui a linha que entrou primeiro pelo bloco da RAM (FIFO - First In First Out), altera o estado para E
 - sim, atualiza a fila de acordo, e a substitui invés de utilizar FIFO e altera o estado para E

Retorna a linha onde foi inserido o bloco, isso para ser utilizada na *escrita()* quando houver necessidade

- **int procuraCopias(int idCache, int *intCopias, int bloco):** do tipo *int*, leva como argumentos a id da cache atual, um ponteiro para o vetor *indCopias* que guarda em suas posições a linha aonde existe uma cópia ou -1 se não existe cópia e a n° do bloco da RAM. Utilizando um *switch case* para verificar nas 2 caches vizinha de acordo com o id passado, verifica se há alguma cópia nelas, se houver acresce a variável *copias* (inicializada com 0), atribui a linha onde encontrou a cópia na posição correspondente em *indCopias[id da cache a ser percorrida - 1]* e faz um *break* encerrando a busca. E então retorna a quantidade de cópias encontradas

Funções Auxiliares:

Funções para impressão de conteúdos e inicializações.

- **void imprimeLinha(int linha, int idCache, int dado):**
imprime a linha passada como argumento
- **void auxConteudoCache(int id):** imprime o conteúdo da cache passada como argumento
- **void auxConteudoRAM():** imprime o conteúdo da RAM
- **int obterOffset(int endereco):** obtém o *offset* do dado no bloco usando o endereço da RAM passado como argumento
- **int obterBloco(int endereco):** obtém o bloco onde se encontra o endereço da RAM passado como argumento

8. CONCLUSÃO

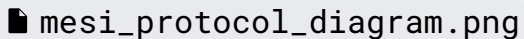
Seguindo o diagrama disponibilizado, a implementação do protocolo MESI e da coerência de cache foi uma tarefa ótima para estudar e entender mais das arquiteturas de computadores e como é funciona a lógica desse gerenciamento.

O programa desenvolvido, ao utilizar *threads* pode possibilitar o incremento de funcionalidades de programação concorrente considerando que como os processadores em uma arquitetura real

executam ao mesmo tempo, as *threads* simulando os processadores podem fazer esse papel em nível de programa. Claro que a inclusão de mecanismos de sincronização como *mutex* e/ou semáforos seriam necessários no caso da implantação de uma aplicação com elas sendo disparadas ao mesmo tempo.

9. REFERÊNCIAS

Conteúdo Classroom de Arquitetura e Organização de Computadores II disponibilizado pela docente Sandra Cossul: <https://classroom.google.com/c/Njc5Njk20TY40Tc2>

Diagrama do protocolo MESI disponibilizado pela docente Sandra Cossul: 

Stallings, William. Computer Organization and Architecture: Design for Performance. PEARSON/PRENTICE HALL, 11ed, 2021.

Patterson, David A. and Hennessy, John L. Computer Organization And Design: The Hardware/Software Interface. MORGAN KAUFMANN, 4th EDITION, 2012.

Patterson, David A. and Hennessy, John L. Arquitetura de computadores: uma abordagem quantitativa. Elsevier, 5ed., 2014.

Shen, J.P.; Lipasti M. H. Modern Processor Design – Fundamentals of Superscalar Processors. Waveland Press, USA, 2013.

